

SQL – Stored procedures e triggers

Este capítulo apresenta procedimentos armazenados (*stored procedures*) e funções (*functions*), procedimentos contendo comandos SQL pré-compilados que são armazenados no banco de dados e podem ser executados pelo usuário. Os gatilhos (*triggers*) são procedimentos contendo comandos SQL, assim como as *procedures*, mas que são associados a alguma operação de *insert*, *delete* ou *update* de alguma tabela. Quando uma dessas operações ocorrer, e esta estiver associada ao gatilho, o procedimento automaticamente será executado.

1 Procedimentos armazenados (*stored procedures*)

Os procedimentos armazenados (do inglês, *stored procedures*) são recursos que combinam lógica de programação e comandos SQL. Segundo Elmasri e Navathe (2018), os procedimentos armazenados são historicamente conhecidos como módulos armazenados persistentes (do inglês, *persistent stored modules* ou SQL/PSM), pelo fato de os códigos estarem armazenados e serem executados pelo SGBD no servidor de banco de dados. A SQL/PSM pode ser entendida como uma linguagem de programação para banco de dados e que estende a linguagem SQL por incluir operadores de atribuição, estruturas condicionais e estruturas de repetição.

Características principais dos procedimentos armazenados:

- São armazenados no banco de dados e podem ser executados por aplicações clientes ou por outros procedimentos ou funções.
- Modularidade, pois permitem construções de módulos pequenos e reutilizáveis.
- Podem ter parâmetros de entrada e saída.
- Permitem executar operações sobre as tabelas do banco de dados como inserções, atualizações e consultas.
- Permitem instruções de atribuição com operações aritméticas, estruturas condicionais e estruturas de repetição.

Segundo Elmasri e Navathe (2018), os procedimentos armazenados são úteis nas seguintes situações:

- Caso o procedimento seja utilizado por várias aplicações, ele pode ser armazenado no servidor. Cada aplicação deve apenas realizar

a chamada do procedimento, passando os devidos argumentos para o procedimento.

- Por estar centralizado no banco de dados, as manutenções sobre o procedimento, tais como alteração, atualização ou mesmo a correção, ocorrem em um só lugar. Assim, todas as aplicações que utilizam o procedimento automaticamente refletem a mudança, sem a necessidade de atualização da aplicação cliente.
- A execução do procedimento diminui o tráfego de dados entre o cliente e o servidor, já que o processamento ocorre no servidor.
- Os procedimentos também podem permitir a verificação de restrições complexas que estão além do alcance das asserções e triggers.

O formato de um procedimento armazenado é dado pelo exemplo 1.

Exemplo 1 – Sintaxe para criar procedimento armazenado

```
CREATE PROCEDURE <nome do procedimento>(<parâmetros>)  
<corpo do procedimento>;
```

Os parâmetros e variáveis locais são opcionais e podem ser declarados, caso seja necessário. Em geral, cada parâmetro deve ser declarado por um tipo de dado SQL. Além disso, os parâmetros podem ser somente de entrada, somente de saída ou de entrada e de saída, que são especificados pela palavra reservada IN, OUT e INOUT, respectivamente. Para executar um procedimento armazenado utilize a instrução CALL (Silberschatz, Korth e Sudarshan, 2020). O formato de execução da instrução CALL é apresentado no exemplo 2.

Exemplo 2 – Sintaxe para a instrução CALL

```
CALL <nome do procedimento> (<lista de argumentos>;
```

Como exemplo, suponha a tabela de funcionários do exemplo 3:

Exemplo 3 – Criação da tabela de funcionários

```
CREATE TABLE FUNCIONARIO (  
    ID INT PRIMARY KEY AUTO_INCREMENT,  
    Nome VARCHAR(100),  
    Cargo VARCHAR(50),  
    Salario DECIMAL(10,2),  
    Departamento VARCHAR(50)  
);
```

Agora suponha uma procedure que será utilizada para inserir um funcionário como apresentado no exemplo 4:

Exemplo 4 – Criação do procedimento inserirFuncionario

```
CREATE PROCEDURE InserirFuncionario (  
    IN p_Nome VARCHAR(100),  
    IN p_Cargo VARCHAR(50),  
    IN p_Salario DECIMAL(10,2),  
    IN p_Departamento VARCHAR(50))  
BEGIN  
    INSERT INTO FUNCIONARIO (Nome, Cargo, Salario, Departamento)  
    VALUES (p_Nome, p_Cargo, p_Salario, p_Departamento);  
END;
```

Os parâmetros do procedimento p_Nome, p_Cargo, p_Salario, p_Departamento são acompanhados pela palavra reservada IN, o que indica que eles são fornecidos pelo usuário no momento da chamada. A procedure insere uma nova tupla na tabela FUNCIONARIO. Para chamar o procedimento utilize o comando CALL como mostra o exemplo 5.

Exemplo 5 – Executando o procedimento inserirFuncionario

```
CALL InserirFuncionario('João Silva', 'Programador', 4500.00, 'TI');  
CALL InserirFuncionario('Maria Souza', 'Vendedor', 7500.00, 'Venda');
```

No exemplo 5, temos a execução do comando CALL para chamar o procedimento inserirFuncionario. Nesse exemplo há duas chamadas, uma para inserir informações do funcionário João da Silva, que é um programador com salário R\$ 4.500 e do departamento de TI. A outra

chamada é para inserir os dados da funcionária Maria Souza que é vendedora com salário R\$ 7.500,00 do departamento de vendas.

Variáveis locais podem ser declaradas por meio do comando declare, como apresenta o exemplo 6, com a sintaxe da declaração de variáveis do tipo varchar. A primeira variável é nome e a segunda é o endereço.

Exemplo 6 – Declaração de variáveis

```
DECLARE nome VARCHAR(50)
DECLARE endereco VARCHAR(50)
```

É possível atribuir valores às variáveis utilizando a instrução SET, como exemplificado a seguir:

Exemplo 7 – Atribuição de valores às variáveis

```
SET nome = 'José da Silva';
SET endereco = 'Rua das rosas, 524. Jardim das Flores';
```

Uma instrução condicional em SQL/PSM pode ser utilizada dentro de uma procedure (Elmasri e Navathe, 2018), a sintaxe é apresentada no exemplo 8.

Exemplo 8 – Estrutura condicional

```
IF <condição> THEN <instruções>
  ELSEIF <condição> THEN <instruções>
  ELSEIF <condição> THEN <instruções>
  ...
  ELSE <instruções>
END IF;
```

O exemplo 8 apresenta a sintaxe da estrutura condicional if, elseif e else. O comando inicia pela palavra reservada if seguida de uma condição que possui um valor lógico que pode ser verdadeiro ou falso, seguido da palavra then. Caso a condição da instrução if seja verdadeira, as instruções serão executadas. Caso contrário, pode ter outras estruturas condicionais definidas pela instrução elseif que também possui condição

associada com instruções atreladas a essa condição. É possível encadear diversas instruções `elseif`. Se nenhuma das condições das instruções `if` ou `elseif` for satisfeita, pode ser tratada com a instrução `else` que não verifica condição nenhuma. Após tudo isso, o comando finaliza com as palavras `end if`.

Já o exemplo 9 demonstra uma procedure que utiliza estrutura condicional, para atualizar o bônus do funcionário. O procedimento recebe o ID do funcionário a quem será aplicado o bônus. Usando o comando `select` obtém-se os dados do funcionário. Agora, verifica-se o departamento que o funcionário trabalha. Se for do departamento de TI, tem o bônus de 10%; caso seja do departamento de vendas, então recebe o bônus de 15%; de qualquer outro departamento, o bônus é de 5%. No final, a procedure atualiza o campo salário baseado no bônus recebido.

Exemplo 9 – Procedure AplicarBonus()

```
CREATE PROCEDURE AplicarBonus (
    IN p_ID INT
)
BEGIN
    DECLARE v_Departamento VARCHAR(50);
    DECLARE v_Salario DECIMAL(10,2);

    -- Obtém o departamento e salário atual do funcionário
    SELECT Departamento, Salario INTO v_Departamento, v_Salario
    FROM FUNCIONARIO
    WHERE ID = p_ID;

    -- Estruturas condicionais para aplicar bônus
    IF v_Departamento = 'TI' THEN
        SET v_Salario = v_Salario * 1.10; -- 10% de bônus
    ELSEIF v_Departamento = 'Vendas' THEN
        SET v_Salario = v_Salario * 1.15; -- 15% de bônus
    ELSE
        SET v_Salario = v_Salario * 1.05; -- 5% de bônus para
        outros
        -- departamentos
    END IF;
```

```
-- Atualiza o salário no banco de dados
UPDATE FUNCIONARIO
SET Salario = v_Salario
WHERE ID = p_ID;
END;
```

A SQL/PSM oferece alguns tipos de estruturas de repetição como o while e repeat (Elmasri e Navathe, 2018).

O exemplo 10 apresenta a sintaxe da estrutura de repetição while. A instrução começa pela palavra reservada while seguida de uma condição que pode ser um valor lógico, verdadeiro ou falso, seguido da palavra DO. O corpo da instrução de repetição pode incluir diversas instruções que podem ser repetidas. O fim do comando é marcado pelas palavras end while. A estrutura de repetição while inicialmente testa a condição, caso seja verdadeira, ela executa as instruções definidas no bloco de instruções.

Exemplo 10 – Estrutura de repetição while

```
WHILE <condição> DO
<instruções>
END WHILE;
```

A estrutura de repetição while realiza o teste no início da instrução. Outra estrutura de repetição pode realizar o teste de condição após a execução das instruções, como é o caso do comando repeat (Elmasri e Navathe, 2018). No exemplo 11, o comando executa as instruções e, somente depois, verifica a condição. Caso a condição seja verdadeira, a estrutura repete as instruções definidas no bloco de instruções. Caso a condição seja falsa, o comando avança para o fim do comando de repetição.

Exemplo 11 – Estrutura de repetição repeat

```
REPEAT
    <instruções>
UNTIL <condição>
END REPEAT;
```

Uma função é semelhante ao procedimento armazenado, mas deve devolver um valor. Para declarar uma função, como consta no exemplo 12.

Figura 12 – Comando CREATE FUNCTION

```
CREATE FUNCTION <nome da função> (<parâmetros>)
    RETURNS <tipo de retorno>
<corpo da função>;
```

Suponha uma função que receba o nome do departamento e devolva o maior salário presente nesse departamento, como descrito no exemplo 13:

Exemplo 13 – Função MaiorSalarioPorDepartamento()

```
CREATE FUNCTION MaiorSalarioPorDepartamento (p_depto VARCHAR(50))
    RETURNS DECIMAL(10,2)
BEGIN
    DECLARE v_MaiorSalario DECIMAL(10,2);

    SELECT MAX(Salario)
    INTO v_MaiorSalario
    FROM FUNCIONARIO
    WHERE Departamento = p_depto;

    RETURN v_MaiorSalario;
END;
```

2 Gatilhos (*triggers*)

Um gatilho (ou *triggers*, do inglês) é um bloco de instruções SQL, pré-compiladas e armazenadas no servidor de banco de dados. Esse conjunto de instruções é executado automaticamente pelo SGBD em resposta a algum evento que modifique o banco de dados (Elmasri e Navathe, 2018).

Segundo Elmasri e Navathe (2018), os gatilhos podem ser necessários em certas condições, que incluem:

- Monitoração do banco de dados dependendo da regra de negócios.
 - Enviar uma mensagem para o supervisor sempre que uma despesa de funcionário exceder um certo valor limite.
- Os gatilhos podem implementar algumas restrições de integridade que não podem ser implementadas usando a restrição de integridade do SQL.
 - Pode-se impedir exclusões de funcionários com um cargo específico (ex.: diretor) sem autorização especial.
- Os gatilhos também podem ser utilizados para auditoria e log de mudanças.
 - Manter os registros de operações críticas como exclusões, inserções ou alteração de algum dado importante.

Segundo Silberschatz, Korth e Sudarshan (2020), para definir um gatilho, deve-se:

- Definir quando o gatilho deve ser executado. Isto ocorre pelo tratamento de um evento disparado pelas operações *insert*, *update* ou *delete*. Em alguns casos, pode ser necessário definir uma condição para definir se o gatilho deve ser executado ou não.

- Especificar a ação, ou seja, qual é o código SQL que deve ser executado para tratar o evento. A ação pode ser executada antes ou depois de uma operação de inserção, atualização ou remoção. A ação pode verificar e comparar valores antigos e novos para os atributos que foram afetados pelo evento que disparou o gatilho, para isso, basta utilizar as palavras reservadas OLD e NEW.

Como exemplo, suponha que se deseje criar um gatilho para registrar no log de auditoria sempre que um novo funcionário for inserido. Vamos ao exemplo 14, em que se cria um gatilho de nome `auditar_contratacao` que será executado com os dados da tabela `FUNCIONARIO`. O gatilho executa após a inserção de um novo funcionário na tabela `FUNCIONARIO`, e executa para cada linha afetada pelo comando de inserção, insere uma tupla na tabela `log_auditoria`. O primeiro atributo da tabela `log_auditoria` contém a descrição do evento, que neste exemplo é o evento de contratação de um novo funcionário. O outro atributo é a data do evento. Após isso, atualiza o atributo `data_contratacao` da tabela `FUNCIONARIO`.

Exemplo 14 – Trigger `auditar_contratacao`

```
CREATE TRIGGER auditar_contratacao
AFTER INSERT ON FUNCIONARIO
FOR EACH ROW
BEGIN
    -- Inserir no log de auditoria o evento de contratação
    do funcionário
    INSERT INTO LOG_AUDITORIA (descricao, data_evento)
    VALUES (CONCAT('Contratação do funcionário ', NEW.nome),
    NOW());

    -- Adiciona a data de contratação na tabela de funcionários
    UPDATE FUNCIONARIO
    SET data_contratacao = NOW()
    WHERE NUMAT = NEW.NUMAT;
END
```

Os gatilhos podem trazer alguns efeitos colaterais, como atualizar tabelas que podem ter, por sua vez, outros gatilhos associados, por exemplo. Esse efeito é um cascadeamento de chamadas de gatilhos que deve ser observado e evitado. O cascadeamento desses gatilhos pode gerar um loop recursivo, pelo fato de um evento disparar um gatilho que, por sua vez, dispara outro gatilho que dispara outro e assim por diante, até que o último gatilho dispara o primeiro, iniciando novamente a cadeia de execuções (Ramakrishnan e Gehrke, 2011).

Considerações finais

Este capítulo apresentou os procedimentos armazenados e as funções, que são blocos de códigos SQL combinados com instruções de lógica de programação e que estão pré-compilados e armazenados no servidor do banco de dados. Esses mecanismos estendem a linguagem SQL permitindo centralizar alguns tipos de operações, diminuindo o tráfego da rede e modularizando algumas regras de negócio, para serem utilizados em clientes diversos e implementados em diversas linguagens de programação.

Em seguida, vimos o conceito de gatilhos, que se constituem também de blocos de comandos SQL combinados com lógica de programação e que podem ser executados em resposta a algum evento que modifique o banco de dados, tais como inserção, alteração e remoção. A seção finaliza com o alerta de evitar o cascadeamento de triggers, pois podem causar efeitos colaterais como o loop recursivo.

Referências

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson, 2018.

RAMAKRISHNAN, Raghu; GEHRKE, Johannes. **Sistema de gerenciamento de banco de dados**. 3. ed. Porto Alegre: AMGH, 2011.

SILBERSCHATZ, Abraham; KORTH, Henry F; SUDARSHAN, S. **Sistema de Banco de Dados**. 7. ed. Rio de Janeiro: GEN LTC, 2020.